IMPERIAL

Department of Mechanical Engineering

Automated Implementation of Boundary Condition Research Using Large Language Models

ME4 Individual Project Final Report

Kyan Jasani-Draper

Supervisor: Prof. Li-Liang Wang

Page Count: 40 (excluding appendices)

Imperial College London

June 5, 2025

Abstract

In industry, constant values are often used in place of accurate boundary condition models due to being difficult and time-consuming to program. This project aimed to bridge this gap through the development of a novel software pipeline to automate the implementation of these models. The pipeline was structured into three main sections: metadata scraping from papers, translation to Python equations, and post-processing. A 770M parameter large language model (LLM) was fine-tuned using parameter-efficient techniques and optimised hyperparameters for the translation section. The model reported a final evaluation loss of 1.272 and a score of 98.6% on a custom evaluation metric designed to check mathematical expression similarity between predictions and ground truth data. Development iterations of the scraping and post-processing sections led to improvements in speed and robustness using efficient API calls and a directed acyclic graph-based equation reducer.

The pipeline was validated on two case studies: a fully algebraic heat transfer coefficient model and a time-dependent friction model. The pipeline deployed the models with success rates of 95% and 35% respectively, demonstrating significant improvements over the naive deployment of the same models carried out by GPT-4o mini (50% and 0% success rates). The pipeline-based approach took 37.72 $\pm$ 1.05 s (SD; n = 20), representing a 96.3% reduction in time compared to the 17 minutes spent manually programming the same model. Lower performance in time-dependent model translation can be attributed to limited LLM size due to computational constraints.

These findings show that an LLM-integrated pipeline can simplify and accelerate boundary condition model deployment, supporting improved data analysis in industry, and providing a framework for wider adoption as translation accuracy improves.

Acknowledgments

I would like to thank my supervisor, Prof. Li-Liang Wang, for his guidance throughout this project and specific knowledge of the state-of-the-art of boundary condition research. I'd also like to thank Vincent Wu for his help in code reviews and for the MathScraper source code.

Contents

1 Introduction **1**
- 1.1 Research Motivation . . . 1
- 1.2 Project Objectives . . . 2
- 1.3 Terminology . . . 2

2 Background & Literature Review **3**
- 2.1 Previous Work . . . 3
- 2.2 The Transformer . . . 3
  - 2.2.1 Tokenizers . . . 6
  - 2.2.2 Training . . . 7
- 2.3 Evaluation & Validation . . . 9

3 Software Architecture **11**
- 3.1 Paper Scraping . . . 12
- 3.2 MathML-Python Translation . . . 13
- 3.3 Post-Processing . . . 15

4 Model Training **17**
- 4.1 Custom Tokenizer . . . 17
- 4.2 Synthetic Data Generation . . . 19
- 4.3 Parameter-Efficient Fine-Tuning . . . 20

5 Results **25**
- 5.1 Case Study I: Interfacial Heat Transfer Coefficient . . . 25
- 5.2 Case Study II: Interactive Coefficient of Friction Modelling . . . 26

6 Discussion **29**
- 6.1 Comparative Study . . . 29
- 6.2 Pipeline Performance . . . 30
  - 6.2.1 Overfitting . . . 30
  - 6.2.2 Model Size Correlation . . . 31
- 6.3 Future Work . . . 32

7 Conclusions **33**
- 7.1 AI-Assisted Writing Declaration . . . 34

References **35**

A Code Example **37**

B Prompting Strategy **40**

1. Introduction

1.1. Research Motivation

Evidence suggests that it takes significant time for leading research in technical fields to be adopted in industry. A study conducted by Salatino et al. in 2020 found an average gap of 5.6 - 6.6 years between academic publications and their corresponding industrial publications/patents in the field of computer science [1]. Although similar quantitative data is scarce, this trend is likely mirrored in the field of boundary condition research, where this project is focused. In this context, boundary condition research refers to the mechanical and thermal principles that describe material interaction, encompassing fields such as tribology (the study of friction and wear between contacting surfaces in relative motion) and heat transfer (the study of thermal energy flow across material interfaces). This field relies heavily on values such as the coefficient of friction (CoF) for the physical modelling of different contact scenarios. Although the CoF is known to vary with temperature, contact force and sliding speed (Figure 1.1), a constant CoF of 0.15 is commonly used for steel in industrial processes [2], demonstrating how a less accurate friction model is applied in industry for ease of use. This is one example of how boundary condition modelling in industry lags behind the most up-to-date research.

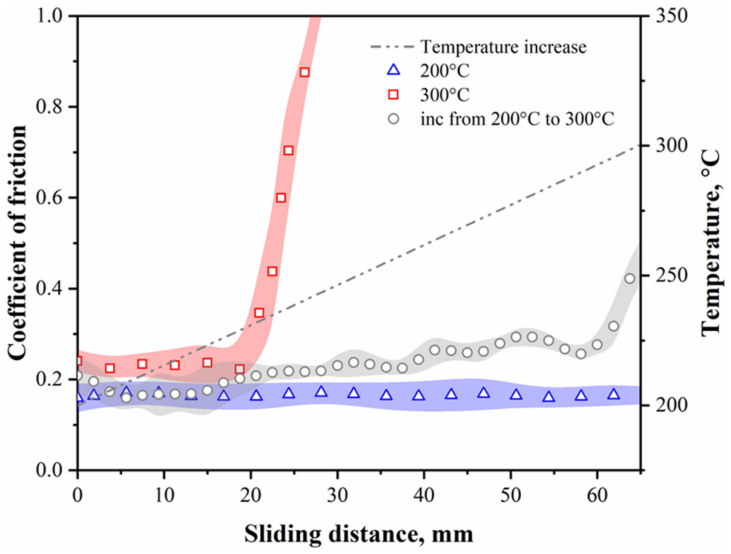


Figure 1.1: Variation of CoF at different temperatures across a set sliding distance [3]. This data is for Aluminium but demonstrates the limitations of using a static CoF value.

A likely reason for this lag is the need for a combination of advanced programming skills and domain knowledge to implement complex models in code. Modern boundary condition models incorporate a system of equations to be solved, often including differential equations and integrals which are time-consuming to set up in code. Empirical constants are also occasionally not stated in papers, leaving it up to the user to apply domain knowledge to estimate a reasonable value.

1.2. Project Objectives

This project aims to simplify the process of applying boundary condition models to industrial data with the help of Large Language Models (LLMs) integrated into a full software engineering pipeline. The detailed objectives are laid out below:

- To build a Python program that fully implements a desired boundary condition model, starting from a paper DOI. The user experience should be simple, requiring little programming ability or domain knowledge, and should speed up the process of model implementation by a factor of 2.
- To have this program outperform the process of inputting a PDF of the paper into a free-access consumer LLM (e.g. GPT-4o mini, Gemini-1.5-flash) and prompting for a code implementation of the model in Python. This improvement may be in the form of better accuracy (avoidance of mistakes in equations), better repeatability (likelihood of correctly modelling the data multiple times in a row) or easier automation.

1.3. Terminology

Table 1.1: Clarifying potentially interchangeable terms that have distinct meanings in this work.

Term	**Definition**
Symbol	A mathematical identifier representing a quantity in a physical model
Constant	A symbol whose value remains unchanged throughout model usage
Variable	A symbol whose value varies during model usage
Parameter	A potentially trainable internal value to a machine learning model (e.g. weights/biases)

2. Background & Literature Review

2.1. Previous Work

MathScraper was a tool developed by Vincent Wu, an Imperial PhD student in the Mechanics of Materials department, as an early attempt to automate the deployment of heat transfer models from papers. The tool makes specific use of MathML, a markup language common in academic literature that uses tags to define different elements of mathematical formulae. For example, the equation $x + y$ is represented in MathML as `<math><mrow><mi>x</mi><mo>+</mo><mi>y</mi></mrow></math>`.

The program uses headless browser automation to scrape the MathML equations, along with relevant constants and model parameters that are stored in tables. The MathML is then converted to LaTeX as an intermediate step, before being converted to a Python implementation of the model. This approach is fully rule-based and relies on the MathML having a consistent structure every time, erroring out with minor syntax variations. With significant manual formatting of symbols and deletion of irrelevant equations, MathScraper is able to implement simple heat transfer models, but consistently fails to parse more complex models with time-dependent features.

The core idea of using MathML as the format to extract the equations, rather than alternatives such as computer vision, was retained in this project. However, the inflexibility of a rule-based approach in adapting to changes in formatting was the primary motivation for adopting an LLM-based approach for the translation of MathML to Python instead.

2.2. The Transformer

To understand how LLMs can be applied to tasks such as the conversion of MathML to Python model implementations, it is helpful to understand how they work. LLMs are built upon the Transformer architecture (Figure 2.1), developed by Vaswani et al. in 2017 [4]. The Transformer is a machine learning architecture initially developed for the purpose of language translation, and has the overall function of converting an input sequence into a series of numerical representations, and then using these to select the most probable output sequence.

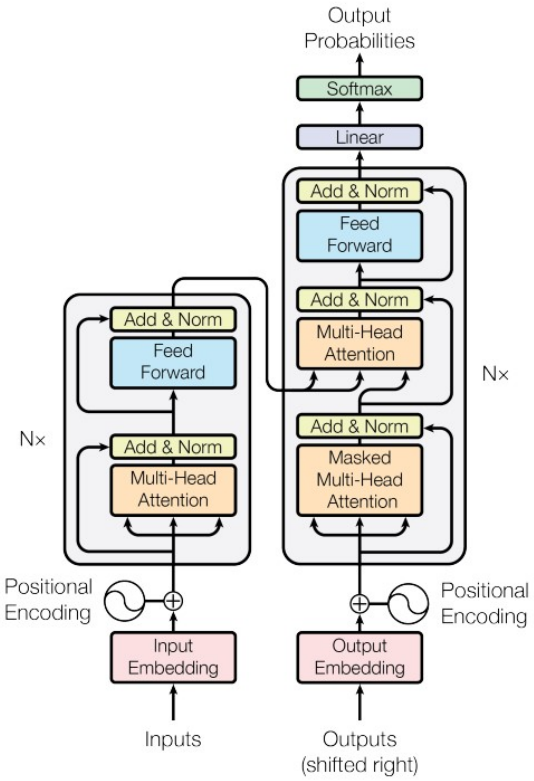


Figure 2.1: The Transformer architecture [4]. The left side represents the encoder block, used to convert the input sentence into context-dependent numerical embeddings. The right side represents the decoder block, used to convert these embeddings into the desired output.

The most effective way to understand how the Transformer improves upon other machine learning architectures for tasks such as translation is by walking through an example, such as translating a short sentence from an input to a target language:

Encoder:

- The input sentence must first be split into distinct sections, or tokens. For this example, each word may represent a different token, although this approach is not commonly used for reasons outlined in Section 2.2.1. An <EOS>, or end-of-sentence, token is also required to end the input.

- Each token is then converted to a constant-size vector of numbers, or embedding, using a learned look-up matrix. This converts a text-based token into a numerical representation that the model can use.

- For each embedding, a position vector is calculated, and the two are summed together. The positional vector represents where a token is in relation to the rest of the input, allowing the model to differentiate between sentences with identical vocabularies in different orders.

- The model now applies the self-attention mechanism in order to numerically represent the relevance of every token in the input sequence to a target token. This score is calculated using the following equation:

$$\text{Attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

  This uses the dot product between the target token (Q) and each other token (K) scaled by the root of the size of the embedding vector (d_k) as a weighting bound between -1 and 1 (due to the use of the softmax function) for the value vector (V). The embedding is used in attention in the prior calculation of the Q, K and V vectors. This mechanism makes the Transformer context-aware, allowing it to differentiate between the meanings of the same token in different usages.

- The attention score has the positional encoding added to it again, and is then normalised before being passed into a feed-forward neural network to increase the Transformer's ability to model complex relationships.

Decoder:

- The output vocabulary is made up of tokens from the target language. Starting from the <EOS> token, the same embedding $\rightarrow$ positional encoding $\rightarrow$ attention scheme is used to predict the next token in the output sequence with one key difference: The self-attention mechanism is masked, meaning only the previously generated tokens can be used to predict the next one to ensure that the model cannot look ahead and use future outputs.

- The decoder-only attention is combined with cross attention between the encoder and decoder to link the significance of words in the input to the predicted output. This cross attention is again normalised, and passed into a feed-forward neural network.

- The output of the final neural network is passed through the softmax function to produce a probability distribution of the next token over the entire output vocabulary. This distribution is sampled from to produce the transformer output sequence.

This demonstrates how LLMs make use of the Transformer architecture to achieve state-of-the-art performance on tasks such as language translation due to their incorporation of context between all tokens in the input sequence and previous tokens in the output sequence.

2.2.1. Tokenizers

As mentioned in the previous section, using entire words as tokens is a simplistic method with many drawbacks. In order to function effectively, LLMs need to be able to handle all words in both the input and target vocabularies. This includes misspellings, variations of words, and unconventional phrases. Including all of these as entire tokens would lead to a vocabulary size on the order of hundreds of thousands or millions [5], drastically increasing embedding size which in turn leads to reduced model performance and efficiency.

Modern language models instead make use of sub-word tokenization in order to break up full words. For example, the word "tokenizer" may be split into "token" and "-izer", while "and" may be retained as a whole-word token due to its frequent use. These sub-word tokens reduce the vocabulary size by multiple orders of magnitude due to reusability, while also allowing the construction of new, out-of-vocabulary words.

Byte-Pair-Encoding (BPE) [5] and Unigram [6] are two sub-word tokenization algorithms often used in LLMs. BPE tokenizers are trained using a frequency-based merging system, where every word is initially split into individual characters, and the frequency of each combination of characters is recorded. The most frequent combinations are merged, and the process repeats a predetermined number of times to create a fixed-size vocabulary, commonly around 30,000 tokens. Unigram tokenizers instead start with a large vocabulary of subwords from the original sequence, assigns each of them a learned probability, and select the segmentation of the input that maximises the overall

likelihood under these probabilities.

The Unigram tokenizer was deemed more suited for the purpose of this pipeline, and an adapted version of it was used for the LLM tokenizer.

2.2.2. Training

Current LLMs have parameter counts in the billions [7] enabled by the use of up to 150 Attention heads and high-dimension feed-forward neural networks. The process of training these models first involves exposure to trillions of tokens of general information and text from the internet with the objective of predicting the next token. This is how all the weights and biases in the feed-forward networks, Q, K and V vectors, and embeddings are set. This type of training, called pre-training, required 16,000 Nvidia H100 GPUs in a full supercomputing cluster for the Llama 3 405B model, making it out of scope for this project. Pre-training equips an LLM with strong language understanding and text-generation ability, but does not provide the model with the capability to perform specific tasks or follow instructions as expected.

The next step in LLM training is supervised fine-tuning in order to improve its performance on specific tasks such as language translation [8]. The model is trained on input-output pairs corresponding to the desired task. The model is first provided the input sequence, then predicts an output sequence. The discrepancy between the predicted output sequence and the true output sequence is computed using a loss function. The gradient of this loss function with respect to each parameter in the model is then used to update these parameters in a process called backpropagation, represented mathematically as follows:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}(f_\theta(x), y) \tag{2.2}$$

where θ is the set of model parameters, η is the learning rate, and $\mathcal{L}$ is the loss function between the predicted output, $f_\theta(x)$, and the true output, y.

Although full supervised fine-tuning needs orders of magnitude fewer training tokens, it still involves updating all model parameters, requiring significant computational time and resources. This

again makes it inefficient for the purpose of this project.

LoRA (Low-Rank Adaptation) is a form of parameter-efficient fine-tuning designed to add additional capabilities to an LLM pre-trained and fine-tuned on similar tasks [9]. Instead of updating each of the billions of model parameters during fine-tuning, LoRA involves freezing all the model parameters so that they don't change during backpropagation and instead updating two smaller sets of parameters. Representing the original frozen model parameters as $W_0 \in \mathbb{R}^{d \times k}$, LoRA specifically involves the introduction and training of BA, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ and $r \ll \min(d, k)$. This allows BA to have the same dimensions as W_0 so they can be summed together to create the full set of model parameters, $W = W_0 + BA$. This is shown in Figure 2.2.

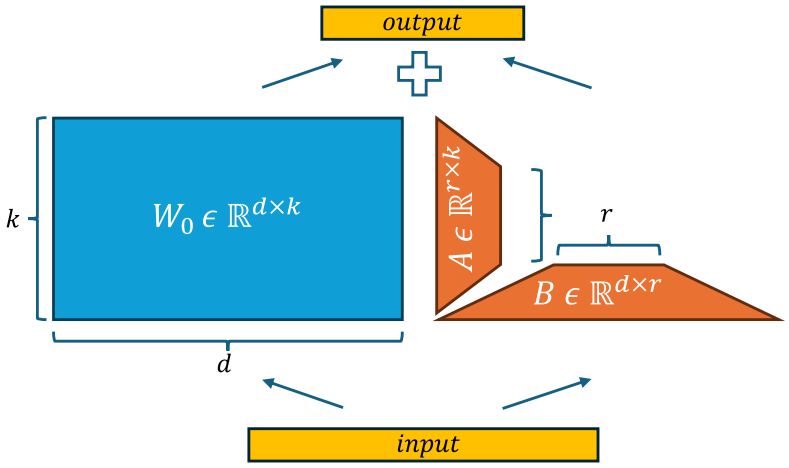


Figure 2.2: The LoRA system for parameter reduction. Frozen model parameters are shown in blue, and additional trainable weights are shown in orange. Adapted from [9].

The rank, r, is a hyperparameter chosen during model training and directly affects the reduction in number of trainable model parameters, reducing from $d \times k$ to $r(d + k)$. As an example, using $r = 8$ allowed Hu et al. to reduce the number of trainable parameters in GPT-3 by 4 orders of magnitude while maintaining or exceeding the performance of a fully fine-tuned model on multiple benchmarks [9]. In this project, LoRA is used in the fine-tuning of an LLM for the conversion of MathML to Python.

2.3. Evaluation & Validation

To determine whether a trained LLM performs well on the task of translating MathML to Python, robust evaluation methods are required. Both evaluation and validation involve the process of passing an input through a model and comparing the output to the corresponding ground truth in order to automate the process of manually assessing the output quality [10]. However, they are distinguished by their purpose and timing: validation is carried out during training to guide model development, while evaluation is done after training on completely unseen data to determine model performance. The evaluation metric is some method of quantifying the distance between the model output and the ground truth.

Accuracy is one of the most common evaluation metrics for numerical tasks like making predictions using neural networks. However, for most generative tasks, such as machine translation, accuracy cannot reliably be used. These tasks are characterised by having a large number of correct outputs that aren't all represented by the ground truth. For example, a MathML to Python translation with ground truth $2x + 2y$ has other equally correct outputs such as $2y + 2x$ and $2(x + y)$. The accuracy metric looks for exact matches and therefore would not take into account any alternative but equally correct outputs, leading to incorrect reports of model underperformance. This drawback can be overcome slightly by providing multiple ground truths to each input, but this is generally impractical for larger datasets, and is still unlikely to cover all semantically identical outputs [10].

BLEU (Bilingual Evaluation Understudy) [11] is one of the most commonly used evaluation metrics for machine translation. Rather than looking for exact string matches between the model output and the ground truth, BLEU evaluates based on the n-gram overlap between the two outputs, where an n-gram is a sequence of tokens of length n. This allows the relative similarity between different outputs and the ground truth to be calculated, rather than assigning a score of 100% accuracy to identical string matches and 0% accuracy to all other outputs. For the example of ground truth $2x + 2y$ and semantically correct but different model output $2y + 2x$, the BLEU score for up to

3-grams with equal weights is:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\frac{1}{3}\sum_{n=1}^{3}\log p_n\right) \tag{2.3}$$

where BP is a brevity penalty to discourage short outputs, and p_n is the modified n-gram overlap.

Assuming character tokenization, the two outputs have 100% 1-gram overlap, 50% 2-gram overlap, and 0% 3-gram overlap. This leads to BLEU ≈ 0.5 using the BLEU Python library, demonstrating an improvement over the accuracy metric that would assign a score of 0. However, there are still clear drawbacks to this metric, demonstrated by the model output still being evaluated as less than perfect. BLEU is generally used for pure language machine translation for a large corpus which differs from the sentence-level translation carried out in this project, a task which BLEU is known to be sub-optimal for [10]. Translating MathML to Python is also not a pure language translation task, but rather a hybrid task that also combines elements of structured code generation, differing again from BLEU's original purpose.

The ideal evaluation metric for the specific task of generating correct Python models from MathML would be one that determines the symbolic accuracy of the output, ensuring that the generated output is mathematically identical to the ground truth rather than just containing overlapping tokens. This is achieved using SymPy, a Python library developed for symbolic mathematics and exact equation manipulation [12]. SymPy provides the tools required to build a custom evaluation metric that determines the equivalence of any mathematical equation to the ground truth. The library also includes methods to check for common formatting errors, such as missing brackets or repeated operators, further improving the effectiveness of the metric. SymPy is used throughout this project for symbolic parsing and evaluation. However, it is important to note that since SymPy only provides a binary assessment of correctness (output is either mathematically equivalent to the ground truth or not), it is not suitable for validation during training. Instead, the standard cross-entropy loss is used during validation as it provides a continuous measure of prediction quality [7] [8].

3. Software Architecture

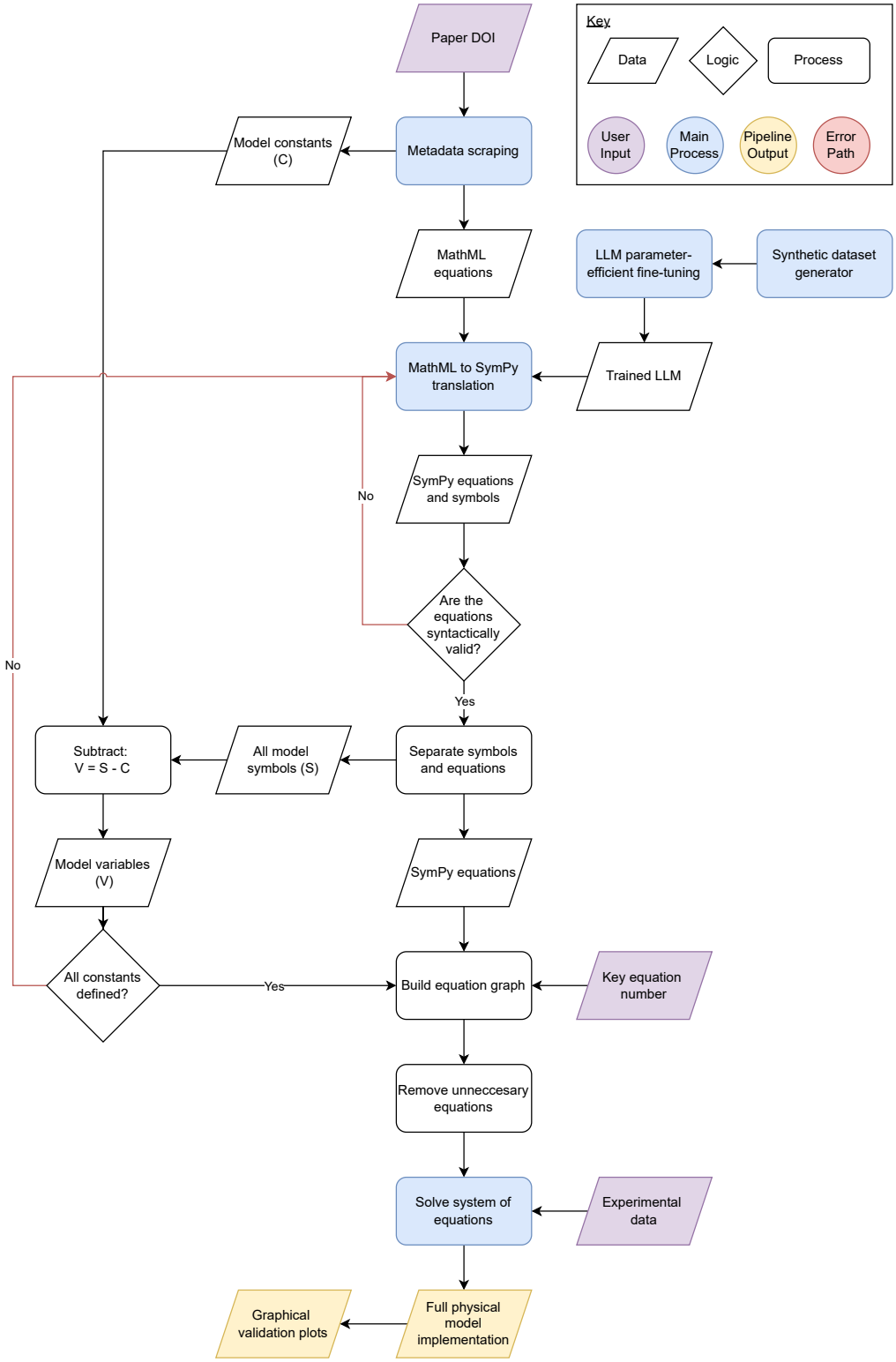


Figure 3.1: Data-flow diagram showing the processes required to convert a research paper into a functional physical model fitted to user data, informing pipeline design.

This project centres around the development of an end-to-end pipeline that takes a paper DOI as input and outputs a full physical boundary condition model that can be applied to a user's dataset, with the architecture defined by the data-flow diagram shown in Figure 3.1. The full codebase can be found in Appendix A.

The design and performance of pre- and post-training sections of the software are discussed in this section, including paper scraping, equation translation, and post-processing. Model training for the purpose of translation is discussed separately in Chapter 4.

3.1. Paper Scraping

The first process of the pipeline is the extraction of the MathML representations of the equations making up a model, along with other necessary constants.

A new approach was deemed necessary after a thorough review of the scraping methods used by MathScraper. The use of a Chrome instance to act as a user accessing the paper online was relatively slow, as it required waiting for the webpage elements to load before the instance could simulate clicking on the link to access the paper. The method also proved unreliable during testing. Attempting to scrape the metadata from the same paper 10 times resulted in 6 successful extractions and 4 content request errors due to timeouts and robot scraping site blockers. The majority of research important to this project is hosted on ScienceDirect, owned by Elsevier, who provide API access to the papers. Normally, this API only exposes front-facing data from the paper, such as the DOI, author, title and abstract. However, using an account with full access to the paper in question allows for the scraping of the full text, including all equations in MathML format along with model parameters in tables. The API scraping method is more reliable and is faster by a factor of 2.4, as demonstrated in Figure 3.2.

After extracting the full text of the paper into Python, the next step is to separate out the MathML equations and model constants. This could be done with string matching, but a more robust method involves the use of the `beautifulsoup` Python library. This library was built to assist in parsing

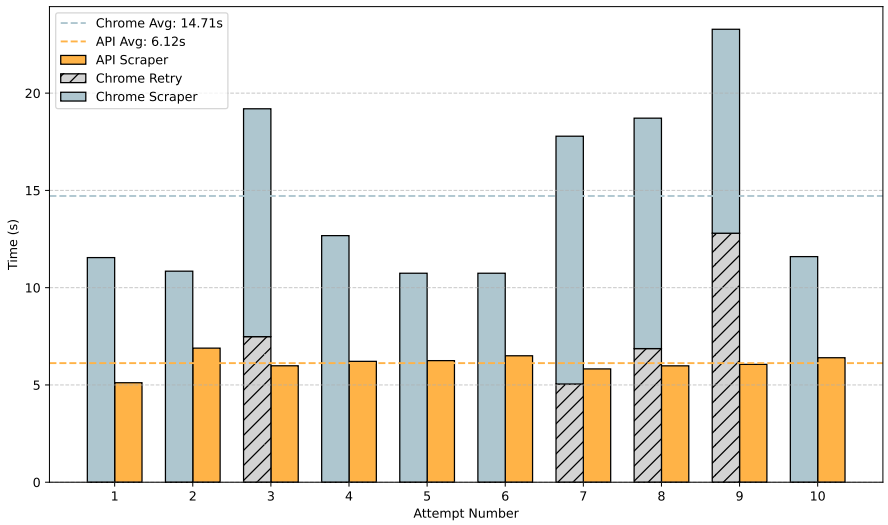


Figure 3.2: Scraping time comparison over 10 attempts using Chrome and API methods. Grey hatched bars indicate Chrome scraping failures that required retry attempts.

XML, a structured and machine-readable form of text storage, and the output of the Elsevier API. The XML is parsed into a tree-like structure that can then be searched for specific structures, such as `<ce:formula>`, which is used to store MathML equations. These equations are extracted and stored as values in a Python dictionary indexed with the equation number for easy access at later stages in the pipeline. The same technique was applied to extracting model constants, alternating between searching table headers for constant names and table bodies for values before being added to another dictionary. Table captions are also searched to ensure they contain words relating to model constants to ensure no excess values are extracted.

After the scraping process is completed, the equation dictionary is passed to the translation section, while the constant dictionary is passed to the post-processing section.

3.2. MathML-Python Translation

The translation section of the software takes in a trained LLM and a dictionary of MathML equations. These equations are then translated into a Python implementation that is initially stored in a string.

A key design decision in this section involved choosing the optimal format for the Python equation implementation after translation and conversion from a string. These equations could either be implemented in raw Python (e.g. `y = 2 * x + 3`) or using SymPy (e.g. `e = Eq(y, 2 * x + 3)`),

and are compared in Table 3.1.

Table 3.1: Decision matrix comparing Raw Python to SymPy equation implementations.

Feature (Weight)	**Raw Python (Score)**	**SymPy (Score)**
Value input (4)	Loop assignment (3)	Internal method pass-through (5)
String-to-code (5)	File write or `exec()` (3)	Easy internal parsing (5)
Safety (5)	Risk of arbitrary code (1)	Very safe handling (5)
Readability (1)	Familiar syntax (5)	Slightly verbose/unfamiliar (4)
Validation (4)	No structural comparison (3)	Structural + syntax checks (5)
Total	**49**	**94**

The decision matrix supported the choice of SymPy equations as the output format of LLM-translated equations due to superior string-to-code conversion, safety, and value inputting ease.

The process of generating the predicted translations was developed in a modular fashion in order to accommodate the variety of models and data input formats used throughout the lifetime of this project. Prediction generation was also carried out using APIs rather than hosting the model locally to bypass the need for local high-performance graphics processing units (GPUs) and large storage systems to hold model weights. The drawbacks of using an API for prediction generation are the possibility of connection and rate limit errors, meaning that retry logic was required in the prediction method. Exponential backoff was used here, meaning that for each subsequent retry n, the system waits for 2^n seconds before attempting to make the prediction again. This was done to reduce the likelihood of repeated collisions with temporary errors, such as network issues.

The `.predict()` method therefore takes in arguments of the model name and number of retry attempts, and outputs the predicted translation of a given MathML equation. This core method was reused in other higher-level methods that provide batch translation functionality for different data formats such as Python lists, .txt and .csv files.

3.3. Post-Processing

The final stage of the pipeline is the post-processing of the SymPy equation strings into a full model implementation. These strings are fully defined with each symbol to ensure validity and simplify symbol extraction. The first post-processing step is therefore to separate the equation and symbol sections of the string using keyword matching and add them to distinct sets, a Python data structure that can only contain one of each element. After parsing all equation strings from a paper, the symbol set will contain both model constants and variables.

Generally, not all equations in a paper will be relevant to the target model. Intermediate equations may be used to show the model's development process, and background theory is often also shown using equations at the start of a paper. Attempting to solve a system of equations formed from all paper equations would typically fail due to reused variables or missing constant definitions. It is therefore crucial for the post-processing section to autonomously reduce the full set of paper equations down to those relevant to the key model. This requires a single user input, the number of the paper's key equation.

The first step in reducing the system of equations to those relevant to the model involves extracting the target variable of the user's chosen equation and removing all other equations that solve for the same variable. This was done with the assumption that all target variables are located alone on the left-hand side (LHS) of an equation which holds true for all papers tested. The second step is to ensure that the remaining system of equations is closed and solvable. Initially, the system of equations was analysed to check that the number of equations was equal to the number of variables (i.e. the system is square) and that all remaining constants had numeric values previously scraped from the paper. However, this method is not sufficiently robust for a fully automated pipeline as it is possible for a system of equations to have an equal number of variables and equations but not be solvable due to containing multiple disconnected subsystems.

To overcome this issue, a more robust graph-based approach was developed and used. In this

context, a graph is a data structure made up of a set of nodes connected by a set of arcs. This project specifically implements a directed acyclic graph (DAG) where each variable is a node, and an arc between node A and node B indicates that B must be known to solve the target equation for A (i.e. A is dependent on B). The directed nature of the graph shows the one-way relationship between target variables and those it depends on, while acyclicity ensures that no variable requires itself to be known to be solved for. An example model dependency DAG is shown in Figure 3.3.

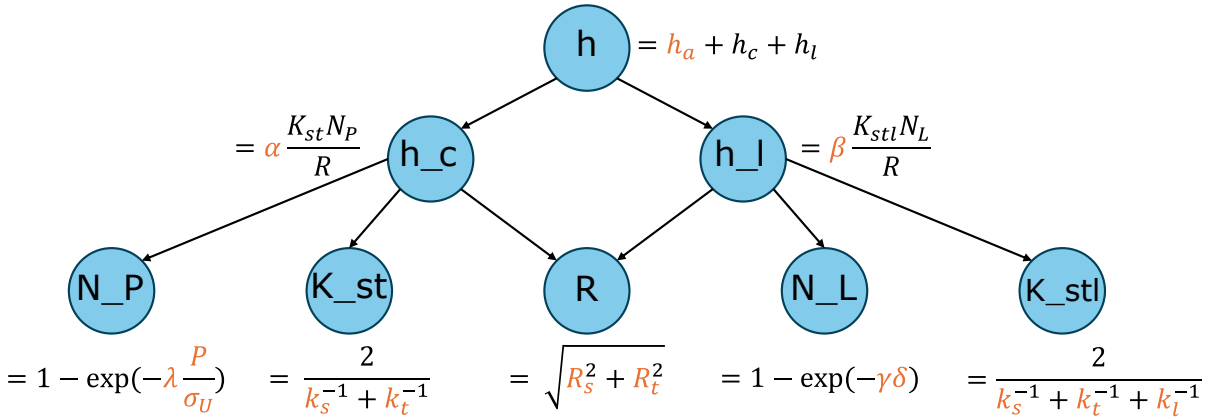


Figure 3.3: Dependency DAG for the target variable h in the interfacial heat transfer coefficient model developed by Liu et al. [13]. Symbols that have no dependencies are shown in orange. This model is solvable, shown by orange symbols only on the lowest level.

The initial DAG for a particular model is built automatically by starting from the target variable, along with a dictionary of model constants, and recursively searching the right-hand side (RHS) of each subsequent equation for all dependencies until a constant is reached. This graph may contain equations unrelated to the target symbol. To reduce this DAG to the dependency DAG, the graph must be searched top-down from the target variable to find the set of all dependencies. This is done using the breadth-first search algorithm which traverses the graph layer by layer to find dependencies. This was chosen over the depth-first search algorithm which showed a tendency to explore chains too deeply and retain irrelevant equations in testing.

After the system of equations has been reduced to contain model-relevant equations only, it can be used to fit the user's dataset and generate predictions that have the potential to be more accurate than the constants and simple models commonly used for boundary conditions in industry. This completes the pipeline, demonstrating the conversion of a paper DOI into a functional model.

4. Model Training

The performance of the pipeline is heavily dependent on the ability of the central LLM to translate MathML equations to SymPy. Training an LLM to carry out this task successfully required the development of supplementary systems to provide labelled task-specific data and the ability for the LLM to reliably convert this unseen data to tokens.

Most well-known LLMs (GPT-4o, Gemini) make use of a decoder-only transformer in their architecture, which is suited for their main task of responding to a user's instruction. The decoder-only architecture doesn't convert the input into an embedding, but instead uses it as a prefix to the output token generation. This architecture is well-suited to user instruction response, where the input prompt can be extremely varied. However, for an LLM trained to carry out a single machine translation task, including an encoder in the architecture was predicted to be beneficial in processing the full input and context.

T5, an encoder-decoder LLM developed by Google for text-to-text tasks such as translation summarisation, was chosen as the underlying pre-trained LLM for this project [8].

4.1. Custom Tokenizer

T5 was originally pre-trained using a Unigram tokenizer on a corpus of text from 4 different languages. This tokenizer shows good generalisation on spoken language text, but since it wasn't trained on MathML, it struggles to efficiently tokenize the tag-based language without breaking up related streams of text. For example, the tag `<mml:mi>` is used frequently to identify mathematical variables in an equation. Due to its frequency and specific purpose, an ideal tokenizer would retain this tag as a single token to preserve context. The original T5 tokenizer does not recognise this and instead splits the tag up into 8 individual characters, completing removing all context and demonstrating the need for a new tokenizer.

Most tokenizer frameworks contains 4 stages: the pre-tokenizer, the normaliser, the tokenizer itself, and the post-processor. A new custom tokenizer was trained using the same SentencePiece

methodology [14] that was used for the original T5 tokenizer, with the function and changes to the 4 stages detailed in Table 4.1. Note that the original T5 tokenizer doesn't use a pre-tokenizer so that it correctly handles languages that separate on spaces (e.g. English) and languages that don't (e.g. Japanese).

Table 4.1: Descriptions and modifications for each stage of the tokenization pipeline.

Stage	**Purpose**	**Original Setup**	**Added Modifications**
Pre-Tokenizer	Split raw text into basic components	None	Splitting on MathML tags (e.g. `<`, `>`)
Normaliser	Clean and standardise text	Lowercasing, Unicode normalisation	Adds Greek letter normalisation
Tokenizer	Segment text into subword units based on learned vocab	Trained on a spoken language corpus	Retrained on MathML/SymPy dataset
Post-Processor	Add model-specific tokens	Adds T5-specific tokens for unknowns and end-of-sentence	No change

The previously outlined tokenization changes were implemented and tested on a dataset of 1,000 examples of MathML to SymPy equation translations to check whether they led to the intended increase in sub-word length (corresponding to fewer tokens per example). The results of this test are shown in Figure 4.1. A reduction in the number of tokens can be seen for the MathML inputs, demonstrating improvement. Comparatively, the distributions for the number of tokens of the SymPy outputs remains similar. SymPy equations strongly resemble spoken language, and mathematical operators such as "Integral" and "Exponential" likely appeared frequently enough in the training corpus of the original T5 tokenizer, explaining its strong performance.

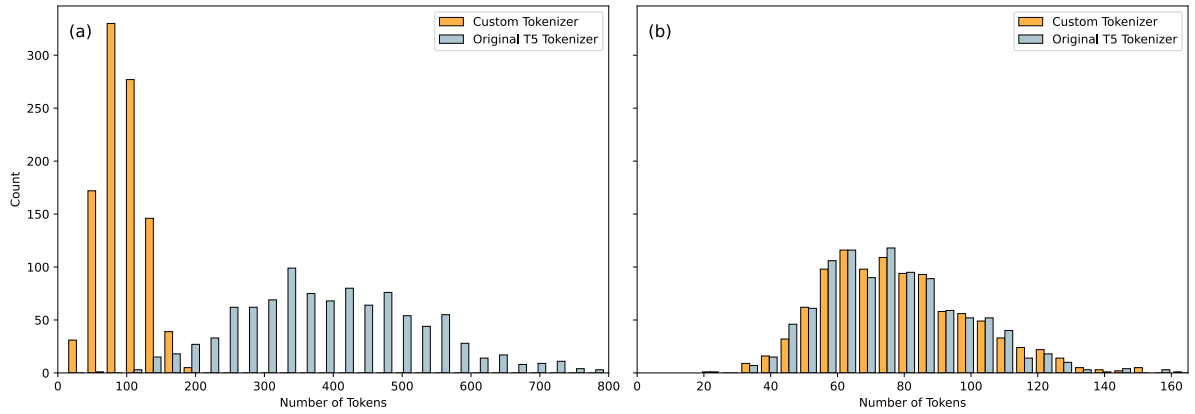


Figure 4.1: Histograms showing number of tokens per example using the original T5 tokenizer (blue) and the new custom tokenizer (orange). (a) shows tokenization of 1,000 MathML input sequences, while (b) shows tokenization of the corresponding Python output sequences.

4.2. Synthetic Data Generation

A large dataset of MathML and corresponding SymPy equations was required for both tokenizer and LLM training. 10,000 examples were deemed the fewest needed for strong training performance. One potential method of creating this dataset would be using the developed scraper to extract the MathML equations from a set of papers and manually add their corresponding SymPy representations to a dataset. However, this approach has significant drawbacks. Extracting and manually translating 13 equations from a familiar paper took 24 minutes. Taking an average of 10 equations per paper, generating a 10,000-element dataset would take an estimated 16.7 days of continuous equation translation, making this method time-consuming and inefficient. Another issue with this procedure was the concern of needing to use some of the desired output models as examples in the training data, leading to potential overfitting and LLM memorisation.

SymPy contains internal methods for the one-way conversion of SymPy equations to MathML. This was used as an alternative method for generating a MathML to SymPy dataset. The Python `random` library was used to generate equations with combinations of different operators, functions, variables and overall types (algebraic, differential equations, integrals and summations). The equations were then converted to MathML, and formatted to match the style used in academic literature. Figure 4.2 shows an information panel relating to a 1,000 equation synthetic dataset.

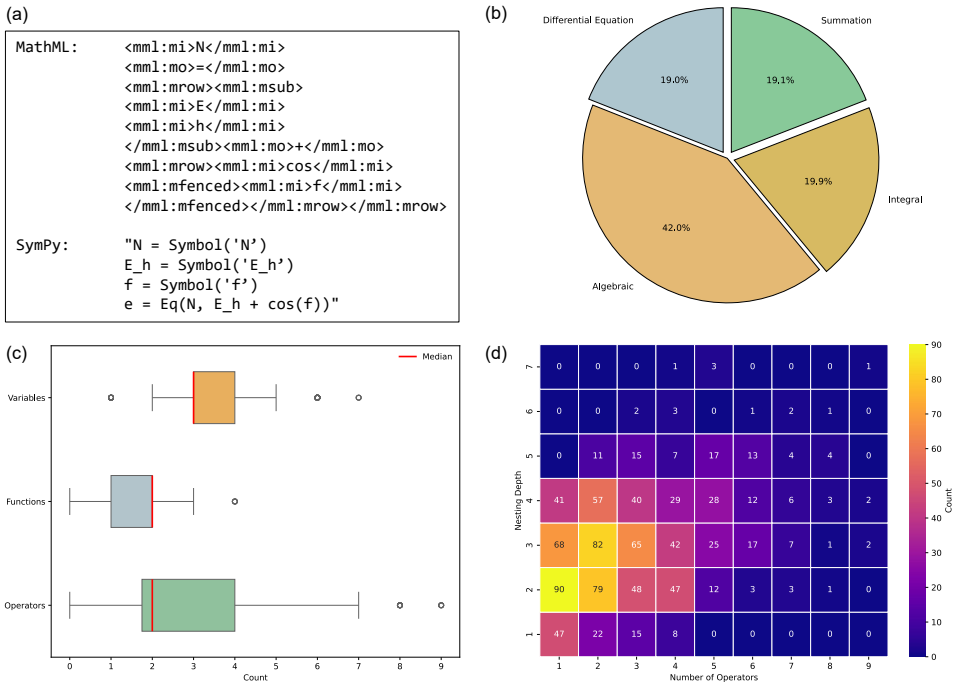


Figure 4.2: Summary panel for the synthetic dataset generator. (a) shows an example entry in the dataset with an automatically generated SymPy equation and its corresponding MathML representation. For a dataset of 1,000 equations, (b) shows the percentage of different equation types, (c) shows the distribution of the number of variables, functions and operators, and (d) shows the joint distribution between operator count and equation nesting depth as a binned heatmap.

The distributions of equation type (Figure 4.2b) and equation features (Figure 4.2c) were both adjusted to replicate the distribution of equations in papers. The correlation between equation nesting depth and number of operators was chosen as a quantitative measure of equation complexity which was again varied until it best matched paper equations.

This equation generator class was incorporated in a parent dataset generation class that allowed for the rapid creation of large equation datasets that could be exported in various file formats. Python multithreading techniques were used to accelerate dataset generation, taking an average of 112 seconds to generate 10,000 examples.

4.3. Parameter-Efficient Fine-Tuning

The T5 architecture is available in several model sizes, each with a parameter count and therefore amount of GPU memory required for training (VRAM). These versions are outlined in Table 4.2.

Table 4.2: Parameter counts and estimated VRAM requirements for the T5 variants. * indicates an estimate.

Metric	**T5-Small**	**T5-Base**	**T5-Large**	**T5-3B**	**T5-11B**
Parameter Count	60M	220M	770M	3,000M	11,000M
VRAM (GB)	3.7	11.8	29.8	45*	90*

For additional context, an average consumer GPU has between 4 and 16 GB of VRAM, while an AI-training-specific NVIDIA A100 GPU can have either 40 or 80 GB of VRAM. This demonstrates the need for an external GPU service in order to train the larger T5 variants. The Imperial high-performance computing cluster (HPC) and Google Colab were trialled as the two main options, with Colab coming out as the stronger choice due to the ability to use the A100 GPU rather than the older A40 through Jupyterhub on the HPC. T5-Large was chosen for training as the largest model that could fit on a single NVIDIA A100.

A dataset of 10,000 MathML to SymPy translations was generated for training, with a further 1,000 examples each for validation and testing. The datasets were augmented with a prefix of "Translate MathML to SymPy:" to match the style of prefix used for the base T5 model before being wrapped in a `huggingface` dataset object. Hugging Face is an open-source Python library developed to assist with LLM training by providing tools for model loading, tokenization and dataset handling [15]. The dataset is then passed through the custom tokenizer to produce a tokenized dataset. Due to the variation of lengths of equations in the dataset, padding is required to make all tokenized examples the same length. This involves adding the -100 token to the end of examples to pad to a preset maximum sequence length.

Once the dataset has been formatted correctly, appropriate hyperparameters must be chosen to ensure LLM training is efficient and effective. For the training of T5, the hyperparameters were split into two distinct categories: LoRA hyperparameters and model hyperparameters. The LoRA

hyperparameters determine the configuration of the parameter-efficient training scheme, while the model hyperparameters control general training behaviour.

The full LLM training configuration contains 11 different hyperparameters. Out of these, the 5 hyperparameters that were mentioned most frequently in LLM training literature were chosen for an optimisation study. These parameters are outlined in Table 4.3.

Table 4.3: Descriptions and value ranges for the five hyperparameters used in the optimisation study.

Hyperparameter	**Description**	**Range**
Learning rate	Controls how much model weights are updated in each backpropagation pass	[5E-5, 1E-4, 3E-4]
Batch size	Number of training examples processed in each forward/backward pass	[8, 16, 32]
Epochs	Number of times the model is trained on the full dataset	[2, 4]
Weight decay	L2 penalty added to the loss function to discourage large weights	[1E-4, 1E-1]
r	Low-rank matrix dimension used in LoRA parametrisation	[2, 4, 8]

By multiplying the length of each range, the size of the hyperparameter search space was calculated to be 108, meaning that 108 full training runs would be required to find the definitive best combination of hyperparameters. This strategy of searching every combination of hyperparameters is called grid search. Improvements on this method can be made by implementing random search and Bayesian optimisation [16]. Random search uses a random selection of hyperparameters for each training run, and is a commonly used technique for narrowing the original search space. Bayesian optimisation is an informed search that uses information about the previous training run to guide the next selection of hyperparameters. The results of a 20-run random search are shown in Figure 4.3.

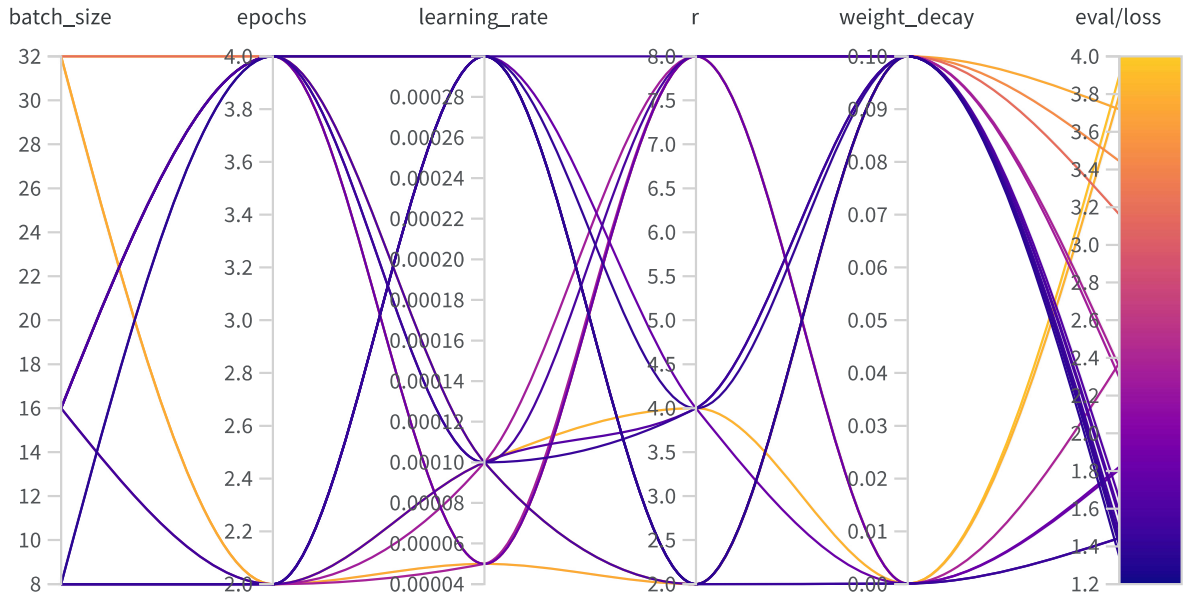
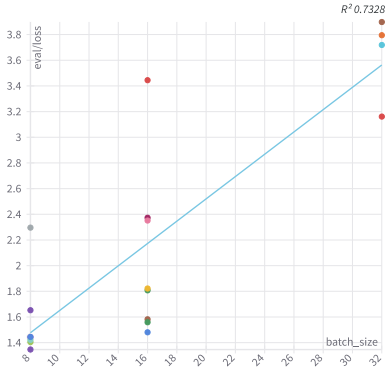
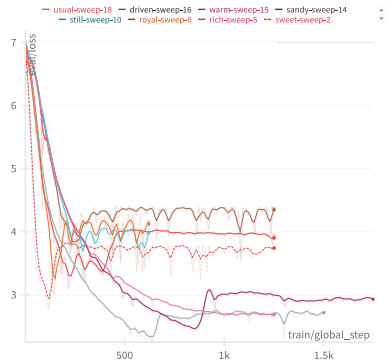


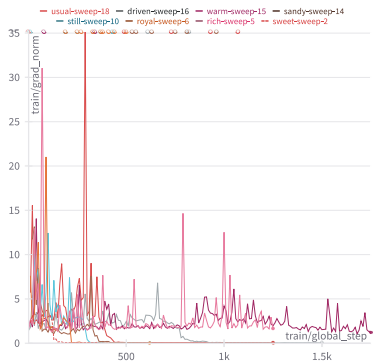
Figure 4.3: Parallel-coordinates plot of a 20-run random search hyperparameter optimisation. Each line represents a single run with hyperparameters defined by intersections with the vertical axes. Runs are coloured by evaluation loss, decreasing from orange (worse) to purple (better).



(a) Scatter plot of batch size against evaluation loss for all runs, showing good positive correlation



(b) Evaluation loss curves for underperforming runs (final evaluation loss > 2)



(c) Gradient norm curves for the same underperforming runs, showing vanishing and exploding gradients

Figure 4.4: Diagnostic plots from the hyperparameter sweep highlighting performance trends.

The importance of hyperparameter selection is demonstrated in Figure 4.3, with evaluation loss varying between 1.35 to 3.98 for the same base model training. This initial study resulted in the following conclusions:

- The batch size hyperparameter showed the strongest correlation with final evaluation loss with an R^2 value of 0.7328 (Figure 4.4a). The majority of high-performing runs (evaluation loss < 2)

come from batch size = 8, which was therefore set as a constant hyperparameter for future optimisations.

- Both exploding and vanishing gradients can be seen in the gradient norm plots for underperforming runs, leading to stalled loss curves (Figures 4.4b and 4.4c). For future optimisations, gradient explosions were mitigated using gradient clipping to scale gradients over a certain threshold. Vanishing gradients are often a result of excessively small learning rates, so the lowest learning rate from Table 4.3 was removed, and 6E-4 was included at the upper range.

These conclusions were used to reduce the search space to 36 hyperparameter combinations which was then optimised using Bayesian techniques. The final model achieved an evaluation loss of 1.272 and an accuracy of 98.6% on the SymPy custom evaluation metric while training 1.6M out of the total 770M parameters.

5. Results

Chapters 3 and 4 showed and discussed component-level results obtained during pipeline development and the training of the central LLM. This chapter moves from individual component-level results to full case studies, evaluating how the pipeline performs at end-to-end model implementation.

5.1. Case Study I: Interfacial Heat Transfer Coefficient

The interfacial heat transfer coefficient (IHTC) is a parameter used to quantify the rate of heat transfer between a hot metal surface and a cooler tool surface in processes such as hot stamping and forming of aluminium panels [13]. Experimental results show that the IHTC varies considerably with contact pressure and lubrication layer thickness, demonstrating the need for an IHTC model rather than making use of constant values. The dependency DAG for this model was shown previously in the Software Architecture chapter (Figure 3.3), representing an 8-equation algebraic model with no differential equations, integrals, or summations. This is one type of model often found in boundary condition research.

The developed software pipeline was tested on its ability to implement the IHTC model from this paper 20 times. The scraping section of the pipeline extracted the MathML equations and model parameters from Table 3 of the paper with a 100% success rate. The translation section was then tested to see the different types of failure modes caught by the retry logic. This is shown in Figure 5.1.

Out of the 20 attempts, 14 were translated successfully the first time, 5 were translated successfully after between 1 and 3 retry attempts, and 1 failed repeatedly, resulting in an overall success rate of 95%.

From the 19 overall successful attempts, the SymPy equations were passed through the pipelines post-processor. To determine whether the equations were translated accurately without disguised errors (slight changes to variable names, flipped signs), the automatically generated plot at the end of the pipeline was compared to the equivalent figure from the IHTC paper, shown in Figure 5.2.

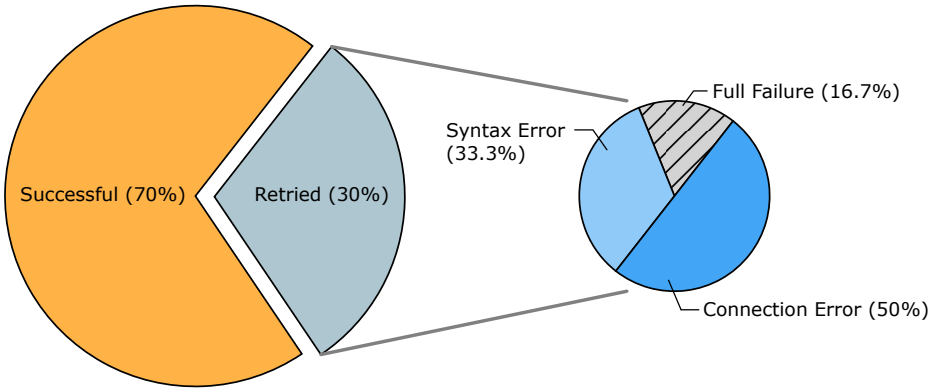


Figure 5.1: Exploded pie chart showing the proportion of successful (orange) and retried (blue) translation attempts of the IHTC model in 20 runs. The failure modes of retried translation are shown in the sub-pie chart on the right in shades of blue, with repeated failure after exhausting all retry attempts shown as the grey hatched segment. 19/20 runs were eventually successful.

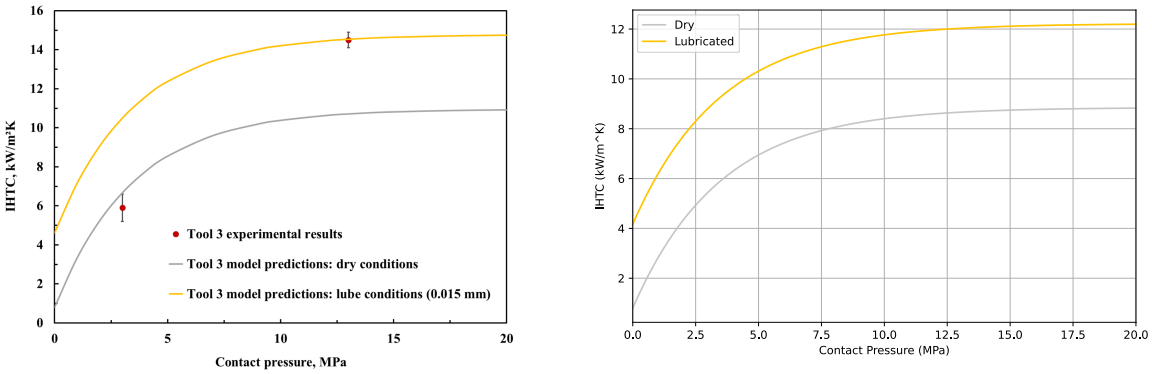


Figure 5.2: Comparison of reference paper plot (left) and pipeline output plot (right) for the IHTC model, used as validation for the correct physical model implementation. Colours and axis legends were added after for visual clarity.

The post-processor accurately implemented all 19 successful translations, demonstrating its robustness, while the graphical validation also reinforces the correctness of the full physical equation model.

5.2. Case Study II: Interactive Coefficient of Friction Modelling

The CoF is another important parameter in processes like hot stamping, and in this context is used to define the resistance to sliding between the tool and the workpiece [3]. The CoF was shown to vary with factors including temperature, sliding speed, and contact pressure through experimental procedures. The main difference between this model and the previously mentioned IHTC model is that the CoF depends on a time-dependent differential equation for the variation of lubricant thickness

with sliding speed. The dependency DAG of this 6-equation time-dependent model is shown in Figure 5.3

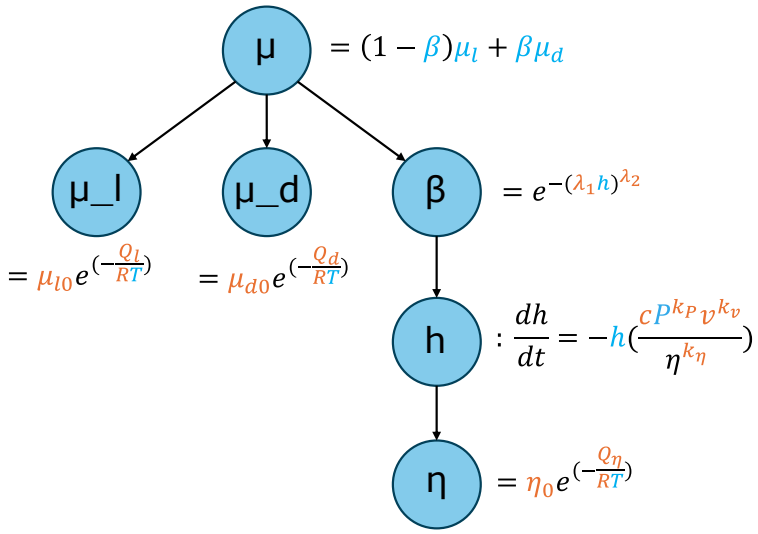


Figure 5.3: Dependency DAG for the target variable μ in the coefficient of friction model developed by Yang et al. [3]. Symbols that have no dependencies are shown in orange, while symbols that vary with time are shown in blue.

The scraping section of the pipeline again extracts MathML equations and model parameters successfully every time. However, the pipeline struggled to translate the more complex MathML of the differential equation to a valid SymPy equation. Out of 20 trials, only 2 were initially successful, with retries resulting in eventual success for 5 more. This led to a much reduced overall success rate of 35%. Looking at the paper model in more detail, a double equals sign was found in Equation 8. With a rule-based translation system, this formatting variation would result in translation failure in every attempt. Therefore, although lower than desired, the 35% success rate demonstrates some ability for the LLM to generalise and handle poorly-structured MathML. This generalisation may be attributed to the large dataset T5 was pre-trained on, or the consistent syntax in the training dataset that the LLM attempted to adhere to even when not strictly present.

The distribution of failure modes for the 18 initially unsuccessful trials also differed significantly compare to the IHTC model. Syntax errors were twice as likely, demonstrating difficulties the trained LLM had in implementing SymPy's differential equation formatting. Examination of individual examples revealed problems in assigning the correct variables in the differential equation according

to SymPy's syntax of `Derivative(dependent_variable, independent_variable)`. Figure 5.4 shows the translation testing results visually.

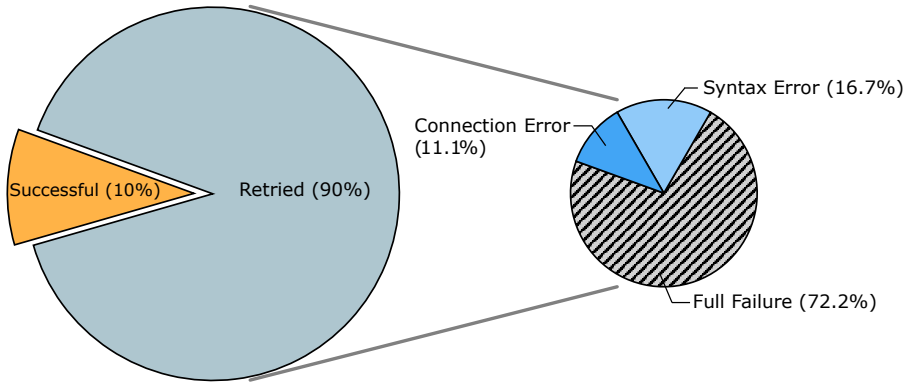


Figure 5.4: Exploded pie chart showing the proportion of successful (orange) and retried (blue) translation attempts of the CoF model in 20 runs. The failure modes of retried translation are shown in the sub-pie chart on the right in shades of blue, with repeated failure after exhausting all retry attempts shown as the grey hatched segment. 7/20 runs were eventually successful.

Initially, passing the full CoF model through the post-processor resulted in consistent failure due to an issue with SymPy's ability to solve the differential equation. SymPy is more focused on analytical mathematics, while this equation requires numerical methods for solving due to the contact pressure term also varying with time. Implementing a forward-Euler solver in the post-processor to accommodate these types of equations led to fully successful model implementation (Figure 5.5, reinforcing the reliability of this section of the pipeline.

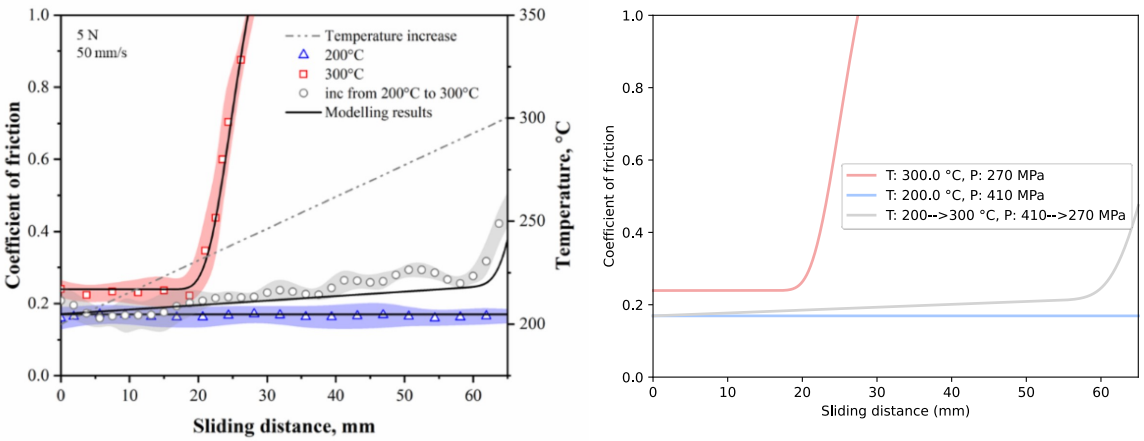


Figure 5.5: Comparison of reference paper plot (left) and pipeline output plot (right) for the CoF model, used as validation for the correct physical model implementation. Colours and axis legends were added after for visual clarity.

6. Discussion

This chapter focuses on the interpretation of results at the case-study level, while also narrowing in on the key sections of the pipeline that have led to these results. In particular, the discrepancies between the success rates of purely algebraic models and time-dependent models are analysed in detail, along with the pipeline's performance on its second objective, demonstrating an improvement over consumer LLMs.

6.1. Comparative Study

At the beginning of this project (October 2024), the most commonly used free-access LLM was GPT-4o mini. This model was used as a baseline to compare the developed pipeline's performance in automatically implementing boundary condition models. The reference model was instructed to implement the two case study papers covered in Chapter 5 in a zero-shot setting, referring to the use of a single prompt and no retry attempts.



(a) Outcome distribution: IHTC model

(b) Outcome distribution: CoF model

Figure 6.1: Comparison of GPT-4o mini implementations of the two case studies, demonstrating reduced performance compared to the pipeline.

The results in Figure 6.1 guided the initial pipeline architecture choices. Unending run times and incorrect model implementations for ODEs demonstrated the need for a programmed post-processor rather than a full LLM-based approach. The use of graphical validation was also chosen as a necessary end to the pipeline from these results. Consumer LLM output code would occasionally look correct and run successfully, but ultimately produce plots that shared no similarity to those from the original paper after further examination. Figure 6.1 also demonstrates the improvements made over consumer LLMs in terms of both accuracy and reliability: overall success rates increased from 50% and 0% to

95% and 35% for the IHTC and CoF models respectively using the pipeline.

6.2. Pipeline Performance

The pipeline takes an average of 37.72s to finish all required processes, with a sample standard deviation of 1.05s across 20 runs. This represents a time saving of 96.3% compared to the 17 minutes spent manually implementing the core 8 equations of the IHTC model (out of the total 24 minutes to implement all equation from the paper). A minimum time saving of 80% is estimated compared to manual implementations carried out by more capable programmers. This demonstrates that the pipeline exceeded its objective of halving model implementation time, allowing for data scientists in industry to easily trial various models in rapid succession.

Performance in the context of accuracy and reliability was more nuanced. The developed pipeline shows strong performance in the end-to-end implementation of algebraic models in the field of boundary condition research, while simultaneously struggling to implement time-dependent models. A difference of 60 percentage points in the success rate demonstrates a significant difficulty in translating time-dependent equations. This result was initially unexpected, as the synthetic dataset that T5 was trained on was specifically designed to contain a proportion of algebraic to time-dependent equations that matched common boundary condition models. Possible causes are discussed below.

6.2.1. Overfitting

One possible concern is that of overfitting, the risk that the LLM has memorised aspects of the training dataset rather than learning to make general predictions. The accuracy score of 98.6% on the SymPy metric is a possible indicator of this phenomenon. Although this score came from evaluation on an unseen test dataset, the data was generated from the same synthetic MathML to SymPy equation generator. This means that the test dataset was unseen to the LLM, but likely contained very similar patterns and structures to the training data, limiting the value of the SymPy metric on assessing the model's ability to generalise. Manually creating a full training dataset from real paper equations was deemed inefficient, but creating a 1,000-example test dataset (approximately 40 hours of work) made up of real equations could lead to improvements in the ability of the SymPy

metric to properly assess the LLM's ability to translate MathML to SymPy in real-world scenarios.

6.2.2. Model Size Correlation

Another possible reason behind the pipeline's underperformance on more complex, time-dependent models relates to difficulties in training larger T5 variants. Before the main training sequence of T5-large, an initial study to ensure the functionality of the Google Colab training environment was conducted. This involved carrying out three training runs of T5-base, the 220 million parameter model size. Despite varying hyperparameters, all runs had similar results: evaluation loss scores > 4, near 0% accuracy on the SymPy metric, and incoherent translations when examining model performance on test data.

The performance differences between T5-base and T5-large in the specific task of MathML to SymPy equation translation mirrors the well-known trend of improved proficiency at most tasks with increasing parameter count [7], [8]. It is therefore predicted that training a larger T5 variant would result in significant performance improvements on time-dependent equation translation. Fully algebraic equations likely resemble the synthetically generated equations enough to be accurately translated, so minimal benefit is expected when moving to a larger model.

With the combined knowledge that performance improves with parameter count and that consumer LLMs perform better at the task of equation translation than equation implementation, a large consumer LLM (Gemini 1.5 Flash) was substituted in for T5 in the equation translation stage of the pipeline before feeding into the same post-processor. Initial performance was subpar due to the large variation in output structure, but incorporating few-shot learning and detailed prompting (Appendix B) following the work of Brown et al. in the GPT-3 paper [17] led to performance comparable to the T5-large model trained on the synthetic dataset. This goes against the original assumption that model architecture would correlate more strongly with pipeline performance than model size; despite the lack of an encoder in Gemini 1.5 Flash, the generalised capabilities of billions of parameters pre-trained on terabytes of data allow for accurate MathML to SymPy translation.

6.3. Future Work

As discussed previously, it is predicted that training a larger T5 variant would lead to stronger pipeline performance in the task of automated model implementation. Fine-tuning a larger model would require a multiple-NVIDIA A100 training setup and a dedicated parallel implementation of the LLM to split the model weights across GPUs. Both the GPT-3 and Llama 3 papers discuss parallelisation techniques which may serve as a guide for future T5 distributed training [7], [17].

Before adding a forward-Euler ODE solver to the post-processor, the pipeline struggled to implement time-dependent models. It is therefore predicted that applying the pipeline to future boundary condition models may require the addition of further custom solvers in order to fit a given dataset using the model. This contradicts the aim of the pipeline to be as autonomous as possible, allowing for industrial use by someone without significant programming ability. One solution would be to pre-empt the potential solvers required, and implement them in the post-processing section before being required. A more elegant solution, and one that is recommended for future work, would be to implement a secondary LLM to recognise the solver needed for a particular model and automatically append it to the solver. LLMs show strong programming skills, especially for single, well-defined functions, so this system likely wouldn't negatively affect the robustness of the post-processor.

Towards the end of this project (May 2025), an arXiv paper focused on the automatic code implementation of machine learning was released [18]. Although the paper's target field differs from that of this project, and their Paper2Code tool failed to implement boundary condition models, the prompting strategy used over fine-tuning to assist with the task of equation extraction and programming seemed promising. Combining the structured prompting approach with the use of a consumer LLM with a parameter count in the trillions as the MathML to SymPy translator could lead to stronger performance at translating time-dependent equations, while maintaining the correct output structure to make use of the same robust post-processor.

7. Conclusions

This project centred around the development of an end-to-end software pipeline for the automatic implementation of models from the field of boundary condition research. This was motivated by the lack of accurate modelling of critical coefficients in industry, where constant values are instead commonly used. This motivation was used to define the main performance of the pipeline: to use an LLM-powered approach to quickly, accurately, and reliably implement models in code, while outperforming the naive use of consumer LLMs.

The pipeline architecture was split into three key sections: paper scraping, equation translation and post-processing. The scraping section uses API calls to extract MathML from papers, a tag-based format used to display mathematical expressions online. This method demonstrates a 2.4 x speed-up factor over headless browser equation extraction, while eliminating failed attempts in a trial of 10 scraping runs.

The equation translation section uses an LLM to carry out the translation of MathML to SymPy, a Python library designed for symbolic computation. T5-large, a 770-million parameter, encoder-decoder architecture LLM pre-trained on language translation tasks was chosen for this purpose. A two-round hyperparameter sweep optimisation was carried out to establish positive correlation with batch size along with detecting vanishing and exploding gradients. The second round of optimisation led to a best model with an evaluation loss of 1.272 and a score of 98.6% on a custom SymPy-based evaluation metric designed to check mathematical similarity of equations.

The post-processor was designed to solve the translated system of equations. Robustness was ensured by using a dependency graph approach to reduce the set of all paper equations down to a solvable system, which performed without issue on the tested papers. With the addition of a forward-Euler solver, the post-processing section was able to execute both algebraic and time-dependent models successfully.

The pipeline was tested at a case study level on two papers: an algebraic model of the interfacial

heat transfer coefficient (IHTC), and a time-dependent model of the coefficient of friction (CoF). Performance on the IHTC model was strong, demonstrating a 95% overall success rate over 20 trials, with the 1 failure coming from repeated mistranslation. The pipeline's ability to fully implement the CoF model was notably weaker, demonstrating a 35% overall success rate over 20 trails, with the 13 failures split between fully failed translations and syntactically valid translations that were only revealed to be incorrect through graphical validation. The pipeline took an average of 37.72s to fully implement a model, representing a 96.3% time saving compared to the 17 minutes programming the model manually.

The pipeline's ability to implement time-dependent models is mainly limited by translation issues stemming from the relatively small LLM parameter count. Future work may focus on either the use of a distributed GPU setup to train a larger T5 variant, or the integration of a trillion-parameter consumer LLM with targeted prompting to eliminate the need for fine-tuning. These approaches are both predicted to improve time-dependent model translation performance, allowing for pipelines like the one developed in this project to become standard tools in the rapid adoption of research for industrial usage in fields including, but not limited to, boundary condition modelling.

7.1. AI-Assisted Writing Declaration

GPT-4o was used sparingly to give suggestions on phrasing for complex sentences. It was also used as a reviewer to rate sections of the work in relation to the provided rubric. The improvements suggested after reviewing were checked carefully and implemented by hand if deemed useful. GPT-4o also proved useful in assisting with LaTeX formatting issues and matplotlib figure development.

References

[1] A. Salatino, F. Osborne, and E. Motta, "ResearchFlow: Understanding the Knowledge Flow Between Academia and Industry," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 12387 LNAI, 2020. DOI: https://doi.org/10.1007/978-3-030-61244-3_16.

[2] L. Muñiz, J. Trinidad, E. Garcia, I. Peinado, N. Montes, and L. Galdos, "On the Use of Advanced Friction Models for the Simulation of an Industrial Stamping Process including the Analysis of Material and Lubricant Fluctuations," *Lubricants*, vol. 11, no. 5, 2023. DOI: https://doi.org/10.3390/lubricants11050193.

[3] X. Yang, X. Liu, H. Liu, D. J. Politis, D. Leyvraz, and L. Wang, "Experimental and modelling study of friction evolution and lubricant breakdown behaviour under varying contact conditions in warm aluminium forming processes," *Tribology International*, vol. 158, p. 106 934, Jun. 2021. DOI: https://doi.org/10.1016/j.triboint.2021.106934.

[4] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, *Attention Is All You Need*, 2017. arXiv: 1706.03762 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1706.03762v1.

[5] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *54th Annual Meeting of the Association for Computational Linguistics, ACL 2016 - Long Papers*, vol. 3, 2016. DOI: https://doi.org/10.18653/v1/p16-1162.

[6] T. Kudo, "Subword regularization: Improving neural network translation models with multiple subword candidates," in *ACL 2018 - 56th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference (Long Papers)*, vol. 1, 2018. DOI: https://doi.org/10.18653/v1/p18-1007.

[7] A. Grattafiori, A. Dubey, A. Jauhri, *et al.*, "The Llama 3 Herd of Models," Jul. 2024. arXiv: 2407.21783 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2407.21783.

[8] C. Raffel, N. Shazeer, A. Roberts, *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," 2020. arXiv: 1910.10683 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1910.10683v3.

[9] E. J. Hu, Y. Shen, P. Wallis, *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR 2022 - 10th International Conference on Learning Representations*, Jun. 2021. [Online]. Available: http://arxiv.org/abs/2106.09685.

[10] M. Fomicheva and L. Specia, "Taking MT Evaluation Metrics to Extremes: Beyond Correlation with Human Judgments," *Computational Linguistics*, vol. 45, no. 3, 2019. DOI: https://doi.org/10.1162/coli_a_00356.

[11] K. Papineni, S. Roukos, T. Ward, and W. J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, vol. 2002-July, 2002. DOI: https://doi.org/10.3115/1073083.1073135.

[12] A. Meurer, C. P. Smith, M. Paprocki, *et al.*, "SymPy: symbolic computing in Python," *PeerJ Computer Science*, vol. 3, e103, Jan. 2017. DOI: https://doi.org/10.7717/peerj-cs.103.

[13] X. Liu, K. Ji, O. E. Fakir, H. Fang, M. M. Gharbi, and L. L. Wang, "Determination of the interfacial heat transfer coefficient for a hot aluminium stamping process," *Journal of Materials Processing Technology*, vol. 247, 2017. DOI: https://doi.org/10.1016/j.jmatprotec.2017.04.005.

[14] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *EMNLP 2018 - Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Proceedings*, 2018. DOI: https://doi.org/10.18653/v1/d18-2012.

[15] T. Wolf, L. Debut, V. Sanh, *et al.*, "Transformers: State-of-the-Art Natural Language Processing," in *EMNLP 2020 - Conference on Empirical Methods in Natural Language Processing, Proceedings of Systems Demonstrations*, 2020. DOI: https://doi.org/10.18653/v1/2020.emnlp-demos.6.

[16] T. Yu and H. Zhu, "Hyper-Parameter Optimization: A Review of Algorithms and Applications," *CoRR*, vol. abs/2003.05689, 2020. [Online]. Available: https://arxiv.org/abs/2003.05689.

[17] T. B. Brown, B. Mann, N. Ryder, *et al.*, *Language Models are Few-Shot Learners*, 2020. arXiv: 2005.14165 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2005.14165.

[18] M. Seo, J. Baek, S. Lee, and S. J. Hwang, "Paper2code: Automating code generation from scientific papers in machine learning," 2025. arXiv: 2504.17192 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2504.17192.

A. Code Example

The pipeline was coded in Python using object-oriented-programming (OOP) principles for modularity. Version control was carried out through GitHub, using branches for feature development and regular commits, pushes and pulls for data management. Figure A.1 shows the DAG class implementation for reference, while the full ≈1500 line codebase can be found at `https://github.com/kyanjd/smart-scraper`.

```
class EquationGraph():
    """
    Encapsulated logic for creating and managing a directed dependency graph of equations
    (nodes = LHS variables, arcs = RHS variables dependent on LHS variables from equations)
    """
    def __init__(self, equations):
        self.equations = equations
        self.graph = defaultdict(set) # Ensures no repeated dependencies
        self.var_equation_map = {}
        self._graph_init()

    def _graph_init(self):
        target_eq = self.equations[0] # Will always be the first equation from earlier
    reinsertion
        self.target = target_eq.lhs.free_symbols # Variable to solve for

        for eq in self.equations:
            lhs = eq.lhs.free_symbols
            rhs = eq.rhs.free_symbols

            for var in lhs:
                self.graph[var].update(rhs) # Add arcs from LHS to RHS variables
                self.var_equation_map[var] = eq

    def BFS_for_vars(self):
```

```
25          """
26          Breadth-first search to find all variables that the target variable depends on
27          """
28          seen = set() # Track visited nodes
29          queue = deque(self.target) # Double-ended queue
30
31          while queue:
32              var = queue.popleft() # Extract the leftmost element
33              if var not in seen:
34                  seen.add(var)
35                  queue.extend(self.graph[var]) # Add all dependent variables to the queue and
    continue search
36
37          return seen
38
39      def get_system_of_equations(self):
40          dependencies = self.BFS_for_vars() # Get all variables the target variable depends on
41          return [self.var_equation_map[var] for var in dependencies] # Return the equations
    needed to be solved for the target variable
42
43      def plot_graph(self, save=False):
44          """
45          Visualise the dependency graph
46          """
47          dependencies = self.BFS_for_vars()
48
49          G = nx.DiGraph()
50          for var in dependencies:
51              G.add_node(var)
52
53          for var in dependencies:
54              for dep in self.graph[var]:
```

```
55                G.add_edge(var, dep) if dep in dependencies else None
56
57        pos = nx.nx_agraph.graphviz_layout(G, prog="dot") # Tree layout
58        plt.figure(figsize=(10, 8))
59        plt.title(f"{self.target} Dependency Graph")
60        nx.draw_networkx(G, pos, with_labels=True, node_size=5000, node_color="skyblue",
    font_size=10, font_weight="bold")
61        plt.axis("off")
62        if save:
63            plt.savefig("Figures/dependency_graph.pdf")
64        plt.show()
```

Listing A.1: Example Python Code

B. Prompting Strategy

For reproducibility, the final prompt following the GPT-3 one-shot guidelines is as follows (inputs to the prompt are shown in curly braces):

```
"You are an expert at translating MathML to Python code using the SymPy library. Translate the following MathML expression into a valid Python SymPy expression.
        Ensure that all variables are treated as symbolic variables and each is defined separately on a new line.
        Assume that SymPy has been imported as ‘from sympy import *‘ so DO NOT include any import statements in your output.
        Example:
        MathML: ’<mml:mi>h</mml:mi>\n<mml:mo>=</mml:mo>\n<mml:mrow>\n<mml:msub>\n<mml:mi>h</mml:mi>\n<mml:mi>c</mml:mi>\n</mml:msub>\n<mml:mo>+</mml:mo>\n<mml:msub>\n<mml:mi>h</mml:mi>\n<mml:mi>g</mml:mi>\n</mml:msub>\n</mml:mrow>’
        SymPy Python code: ’h = symbols(’h’)\nh_g = symbols(’h_g’)\nh_c = symbols(’h_c’)\neq = Eq(h, h_g + h_c)’

        Now, translate the following MathML expression into Python code using SymPy.
        MathML: {mml}
        SymPy Python code:"
```

Listing B.1: Final Prompt